

Your Health Records, On Your Terms.

A 30-minute quickstart for the private,
agent-mediated personal health stack.

" You own your records. The cloud is just a courier. "

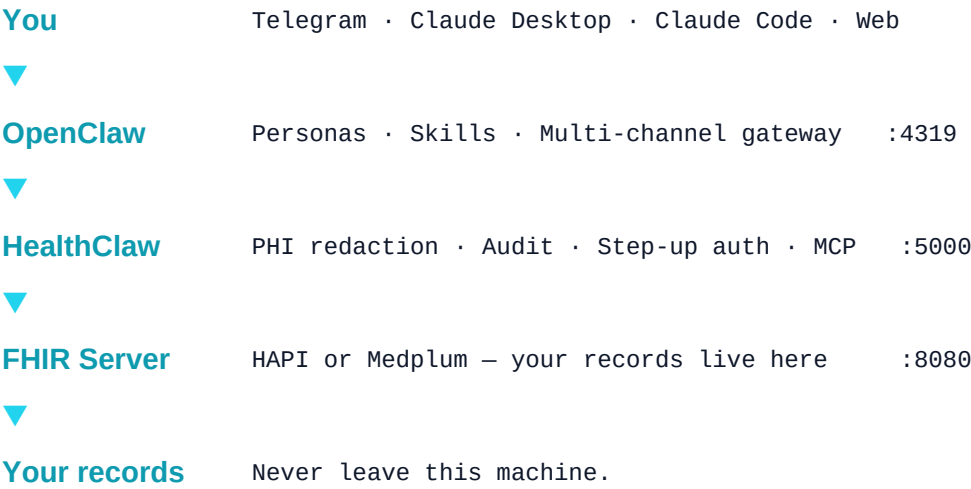
Edition · 2026-05-05
github.com/aks129/HealthClawGuardrails



What you're about to build

By the end of this guide your machine — laptop, Mac mini, Linux box — will hold a complete, private, agent-mediated view of your own clinical records. Nothing leaves the box unless you say so.

The stack, top to bottom:



Privacy guarantee

Records flow EHR → connector → your machine and stop there. PHI redaction is applied **in-process** before any file is written. The only outbound traffic during data pulls is the OAuth/SMART-on-FHIR handshake your EHR requires.

Prerequisites

macOS / Linux	<code>uname -sm</code>
Python 3.11+	<code>python3 --version</code>
Node 22+	<code>node --version</code>
Docker (optional)	<code>docker --version</code>
git	<code>git --version</code>
~30 minutes	—

Install OpenClaw

OpenClaw is your local AI gateway. It runs the agent personas, exposes them on whichever channels you want — Telegram, WhatsApp, iMessage, Slack, web — and gives them access to your installed skills.

Already installed?

```
which openclaw
openclaw --version
ls ~/.openclaw 2>/dev/null
```

If those return cleanly, skip ahead.

Fresh install

The canonical install lives at openclaw.ai. Use whatever the homepage says — install method changes between releases. Historical commands the repo has used:

```
# A – npm (most common)
npx -y @openclaw/cli init

# B – Homebrew tap
brew install openclaw/tap/openclaw

openclaw auth login          # OAuth in browser
openclaw gateway start      # http://localhost:4319
openclaw status              # confirm running
```

Tip. For an always-on Mac mini setup with LaunchAgent + caffeinate, see [docs/mac-mini-setup.md](#) in the HealthClaw repo.

Stand up an open-source FHIR server

Your records need somewhere to live. Two solid choices — pick one.

Option A — HAPI FHIR · recommended

Reference Java implementation. Easiest to run, no auth on localhost, perfect for first-timers.

```
# Already up?
curl -sf http://localhost:8080/fhir/metadata >/dev/null && echo OK

# Otherwise — Docker is fastest
docker run -d --name hapi-fhir \
  -p 8080:8080 \
  hapiproject/hapi:latest

# After ~30s
curl -s http://localhost:8080/fhir/metadata | head -c 200
```

Option B — Medplum · more features, OAuth-ready

Full Medplum platform: FHIR R4 + auth + dashboard + audit. Heavier setup but gives you a UI to browse resources.

```
git clone https://github.com/medplum/medplum
cd medplum
docker compose -f compose-dev.yaml up -d
open http://localhost:3000 # admin UI
```

Public sandboxes exist (hapi.fhir.org, r4.smarthealthit.org) but **do not put real records there** — they're shared, wiped weekly, and visible to anyone.

Connect your real records (privacy-first)

Authenticate with your providers and pull records into the FHIR server you just stood up. Records flow EHR → connector → your machine. The cloud is only the OAuth provider.

Pick a connector

Service	Best for	Auth
HealthEx	US Epic / Cerner / CommonWell users — easiest	OAuth
fhir-skills (Mandel)	DIY SMART-on-FHIR savvy users	SMART
Flexpa	Apps that want a hosted FHIR endpoint	OAuth
Direct SMART	Power users who want zero middleware	OAuth+PKCE
TEFCA IAS / Fasten	Cross-network records via QHIN	Stitch

Recommended path — HealthEx

Lowest friction, cleanest output, first-class HealthClaw integration via the `/export` slash command.

```
# 1. Sign up at healthex.io
# 2. claude.ai → Settings → Integrations → HealthEx → Connect
# 3. Connect your health systems (Epic, Cerner, etc.)
# 4. Stash your token in macOS Keychain so agents find it
security add-generic-password -s healthex -a me -w '<your-token>'
```

Install HealthClaw Guardrails

HealthClaw sits between any agent and your FHIR server, enforcing PHI redaction, immutable audit trails, step-up auth on writes, and tenant isolation on every request.

```
git clone https://github.com/aks129/HealthClawGuardrails
cd HealthClawGuardrails
uv sync || pip install -e .

cp .env.example .env
# In .env:
# STEP_UP_SECRET=$(python3 -c 'import secrets; print(secrets.token_hex(32))')
# FHIR_UPSTREAM_URL=http://localhost:8080/fhir # HAPI from Step 1b

docker-compose up -d --build # full stack (Flask + Redis + MCP)
curl http://localhost:5000/r6/fhir/health
```

Wire the OpenClaw personas

One script seeds Sally (PCP), Mary (pharmacy), Dom (fitness), and Kristy (scheduler) — each persona's AGENTS.md already lists every HealthClaw slash command and tool.

```
./scripts/seed_openclaw_workspaces.sh
./scripts/update_agent_prompts.sh
./scripts/bot_commands_install.sh # installs ~/.healthclaw/commands.py
```

Connect Claude Desktop / Code via MCP

```
{
  "mcpServers": {
    "healthclaw-local": {
      "type": "http",
      "url": "http://localhost:3001/mcp",
      "headers": { "X-Tenant-ID": "my-health" }
    }
  }
}
```

Pull data through HealthClaw

Three ways. Pick whichever fits your moment.

A · One slash command from any bot

DM Sally / Mary / Dom / Kristy on Telegram:

```
/export # HealthEx → redact → file
/import ~/.healthclaw/exports/healthex-<date>.json # bundle → FHIR via guardrails
```

B · Run the scripts directly

```
HEALTHEX_AUTH_TOKEN="$(security find-generic-password -s healthex -w)" \
python scripts/export_healthex_mcp.py \
--tenant-id my-health \
--output exports/healthex-$(date +%Y-%m-%d).json

python scripts/import_healthex.py \
--bundle-file exports/healthex-$(date +%Y-%m-%d).json \
--tenant-id my-health \
--step-up-secret "$STEP_UP_SECRET"
```

C · Conversationally via Claude

If HealthEx is connected to claude.ai, just ask:

```
Pull my complete health history across all categories going back
15 years, fully paginated. Build a de-identified FHIR R4 transaction
bundle with US Core resources and write it to
healthclaw-bundle-<date>.json.
```

What gets redacted. *HumanName* → initials, *Address* → state+country, *Identifier* → SHA-256 hash, *birthDate* → year, *telecom* → ***, *narrative div* → empty. Clinical codes (ICD-10, SNOMED, LOINC, RxNorm, CVX) are **preserved** — they're the signal you actually want to analyze.

Verify everything works

A 60-second checklist. Every line should green-light.

Liveness

```
openclaw status # → running
curl -sf http://localhost:8080/fhir/metadata | head -c 80
curl -sf http://localhost:5000/r6/fhir/health
curl -sf http://localhost:3001/mcp/health || true # MCP orchestrator
```

Records present

```
curl -s "http://localhost:8080/fhir/Patient?_summary=count" \
  | python3 -c "import sys,json; print('Patients:', json.load(sys.stdin).get('total'))"
```

Agent smoke test

In Claude with the healthclaw-local MCP server connected:

Use the healthclaw-local tools. Get a step-up token, search for all Patients in tenant my-health, then read one Condition and one Observation. Confirm responses include _mcp_summary and that PHI looks redacted.

Telegram smoke test

DM any persona:

```
/health
/summary
/conditions
```

Each should return a structured response paraphrased by the LLM. Names appear as initials. Identifiers prefixed with redacted:sha256:. AuditEvents recorded for every read.

Where to go from here

HealthClaw ships with eight specialised skills that go deeper than this guide. Loaded into Claude Desktop / Code, they trigger on the matching prompts.

getting-started	This guide. End-to-end onboarding.
fhir-r6-guardrails	The 14 MCP tools and their guarantees
phi-redaction	Exactly what gets redacted and why
fhir-upstream-proxy	Wire HAPI, Medplum, Epic upstream
personal-health-records	Conversational HealthEx data pull workflow
healthex-export-redacted	MCP-SDK + in-process PHI redaction (the /export path)
healthex-export	Tenant-to-tenant copy inside HealthClaw
curatr	Find data quality issues; propose patient-approved fixes
fasten-connect	TEFCA IAS · Stitch widget · QHIN cross-network

Browse all skills at healthclaw.io/skills · or in the repo at github.com/aks129/HealthClawGuardrails/tree/main/skills.

[HealthClaw](#) · [OpenClaw](#)
Open source · MIT License · A healthclaw.io project